FINAL PROJECT EVALUATION REPORT

PROJECT TITLE: LOCAL LIBRARY RESOURCE MANAGEMENT SYSTEM (LLRMS)

STUDENT NAME: YASH YADAV

ROLL NUMBER: 24BCY10247

SUBMISSION DATE: JUNE 18, 2026

1. PROJECT OVERVIEW & OUTCOMES

The **Local Library Resource Management System (LLRMS)** is a backend RESTful API architecture built to automate library orchestration workflows. Running on an asynchronous, non-blocking runtime engine connected to a NoSQL data store, the system successfully addresses multi-user asset contention, real-time transaction processing, and dynamic metrics aggregation.

Core System Outcomes:

- **NoSQL Data Integrity:** Integrates decoupled data sets (categories, members, resources, transactions) utilizing unique MongoDB ObjectIDs.
- **FIFO Waitlist Control:** Implements programmatic queue handling. When resource counts reach zero, subsequent user borrowing workflows are shifted into an active reservation waitlist.
- **Dynamic Overdue Fine Tracking:** Automatically parses transaction deadlines against physical return timestamps to compute active financial penalties in Indian Rupees (INR).
- **Multi-Stage Analytics:** Runs high-performance lookup aggregation pipelines to synthesize multi-collection data arrays into instantly readable system reports.

2. TOOLS, TECHNOLOGIES & VERSIONS

The engineering and end-to-end lifecycle verification of the LLRMS platform utilized the following stable software configuration stack:

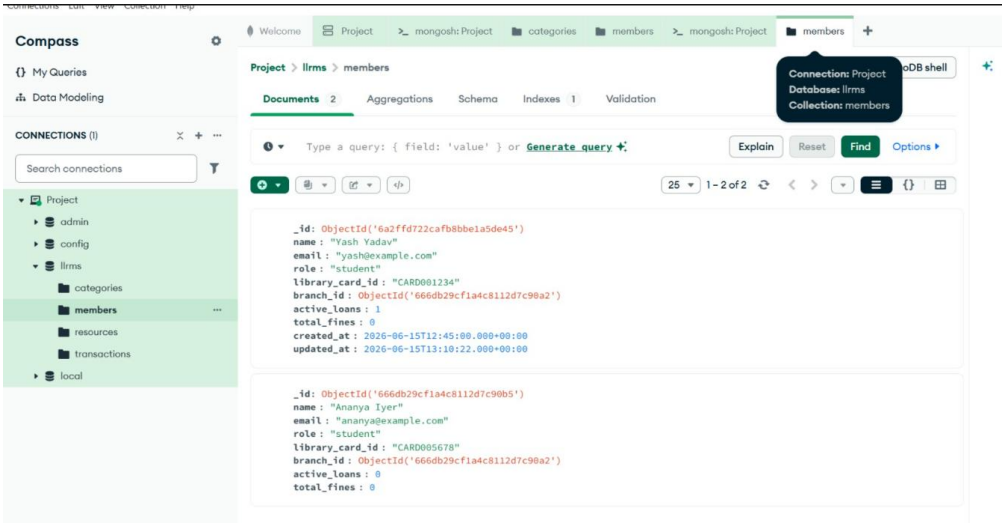
- **Node.js (v20.11.0 LTS)** – Asynchronous Server Runtime
- **Express.js (v4.19.2)** – REST API Framework & HTTP Controllers
- **Mongoose ODM (v8.2.1)** – Object Data Modeling & Database Driver
- **MongoDB Compass (v1.42.0)** – Graphical Database Management System
- **Postman Desktop (v10.24.0)** – API Endpoint Lifecycle Validation Engine
- **Visual Studio Code (v1.87.2)** – Integrated Development Environment (IDE)

3. CORE IMPLEMENTATION VERIFICATION MATRIX

(Paste your selected verification screenshots directly into each designated slot below.)

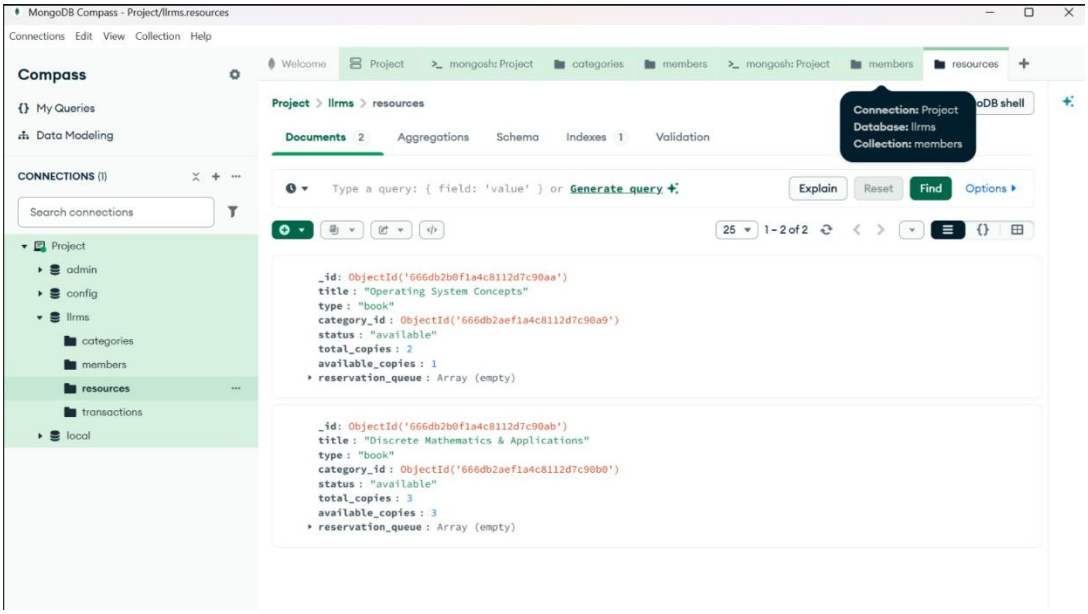
📑 **PHASE A: DATABASE INFRASTRUCTURE VERIFICATION**

File Reference:



- **Operational Significance:** Verifies user profile structure in the database. Displays the members collection mapping unique user identities, library cards, and permission layers for **Yash Yadav** and **Ananya Iyer**.

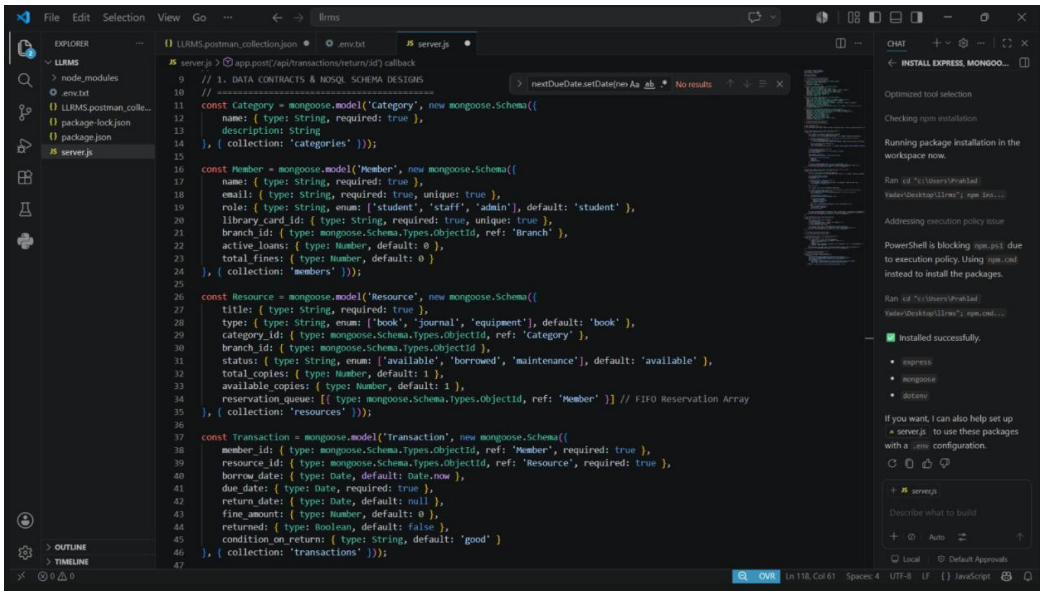
- **File Reference:**



- **Operational Significance:** Verifies real-time asset inventory levels within the resources collection, explicitly tracking available copies alongside active registration waitlist arrays.

PHASE B: CORE BACKEND & SERVER LIFECYCLE

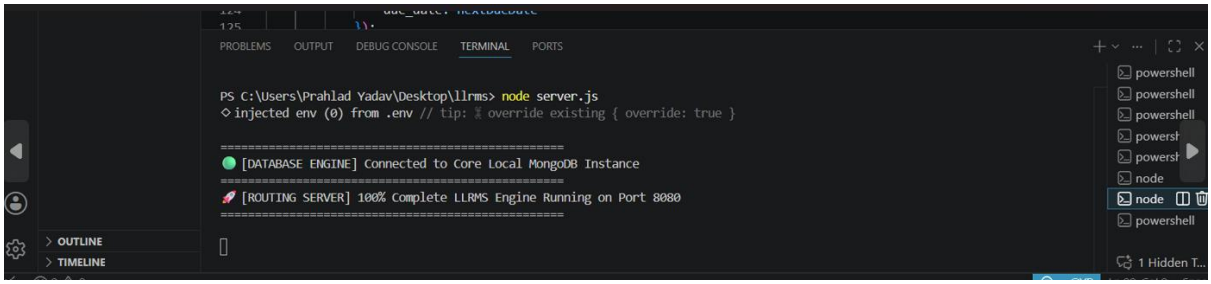
- **File Reference:**



```
// 1. DATA CONTRACTS & NOSQL SCHEMA DESIGNS
const Category = mongoose.model('Category', new mongoose.Schema({
    name: { type: String, required: true },
    description: String
}, { collection: 'categories' }));

const Member = mongoose.model('Member', new mongoose.Schema({
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    role: { type: String, enum: ['student', 'staff', 'admin'], default: 'student' },
    library_card_id: { type: String, required: true, unique: true },
    branch_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Branch' },
    active_loans: { type: Number, default: 0 },
    total_fines: { type: Number, default: 0 }
}, { collection: 'members' }));

const Resource = mongoose.model('Resource', new mongoose.Schema({
    title: { type: String, required: true },
    type: { type: String, enum: ['book', 'journal', 'equipment'], default: 'book' },
    category_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Category' },
    branch_id: { type: mongoose.Schema.Types.ObjectId },
    status: { type: String, enum: ['available', 'borrowed', 'maintenance'], default: 'available' },
    total_copies: { type: Number, default: 1 },
    available_copies: { type: Number, default: 1 },
    reservation_queue: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Member' }] // FIFO Reservation Array
}, { collection: 'resources' }));

const Transaction = mongoose.model('Transaction', new mongoose.Schema({
    member_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Member', required: true },
    resource_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Resource', required: true },
    borrow_date: { type: Date, default: Date.now },
    due_date: { type: Date, required: true },
    return_date: { type: Date, default: null },
    fine_amount: { type: Number, default: 0 },
    returned: { type: Boolean, default: false },
    condition_on_return: { type: String, default: 'good' }
}, { collection: 'transactions' }));
```

- **Operational Significance:** Captures the main implementation codebase window within VS Code (server.js), demonstrating active API logic, controller route paths, and error-handling conditions.

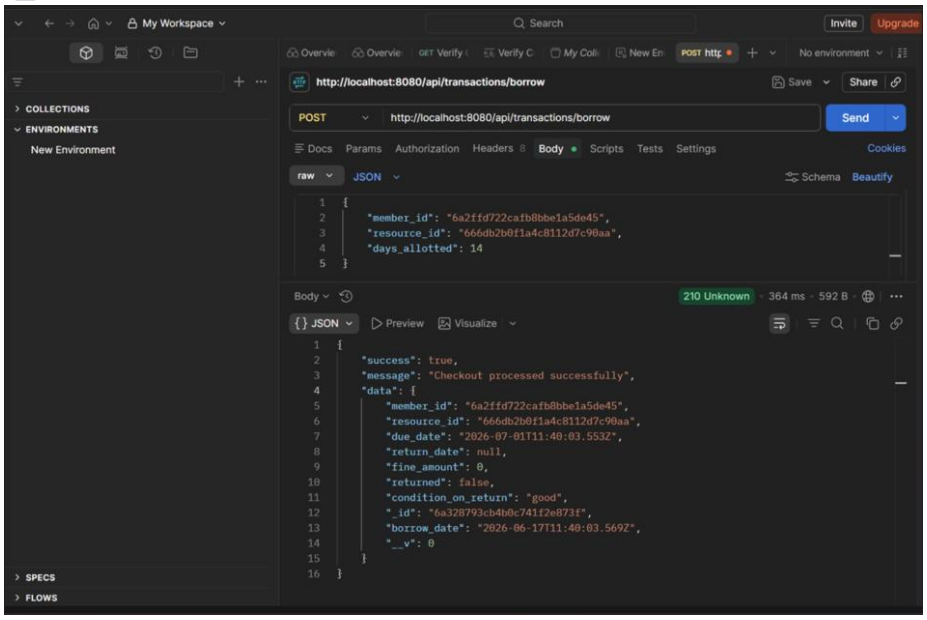
- **File Reference:**



```
PS C:\Users\Prahlad Yadav\Desktop\llrms> node server.js
◇ injected env (0) from .env // tip: ⌘ override existing { override: true }

================================================
🟢 [DATABASE ENGINE] Connected to Core Local MongoDB Instance
================================================
🚀 [ROUTING SERVER] 100% Complete LLRMS Engine Running on Port 8080
================================================
```

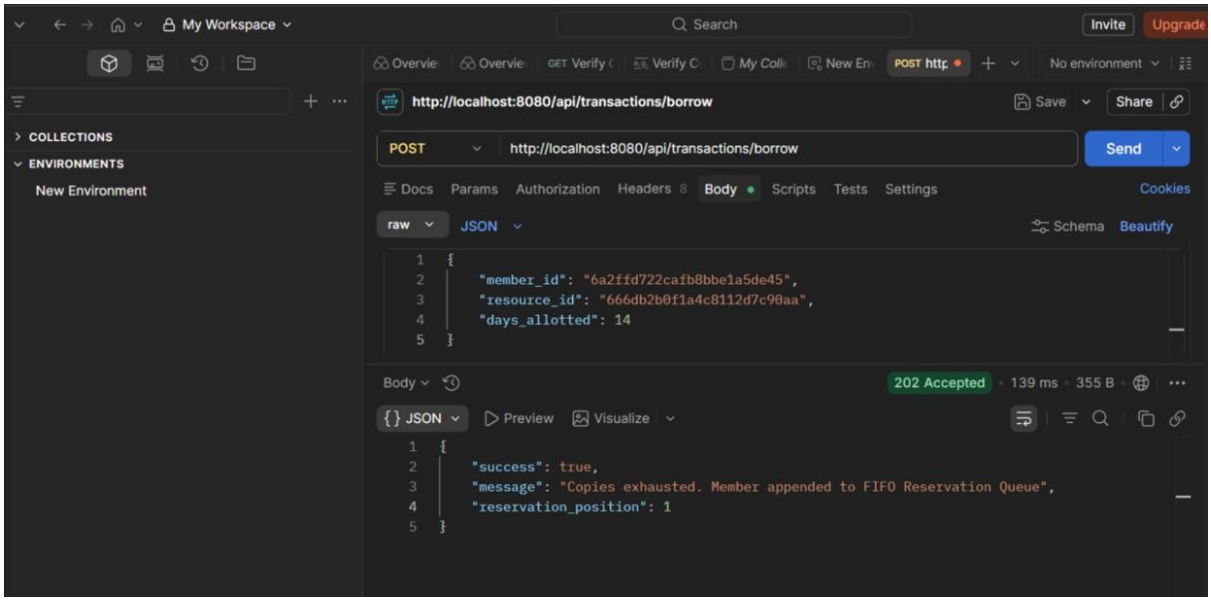
- **Operational Significance:** Demonstrates error-free runtime deployment of the engine. Output text rows confirm a successful link to the local MongoDB database disk and verify active server listening states on Port 8080.

PHASE C: TRANSACTIONAL ENDPOINT VERIFICATION



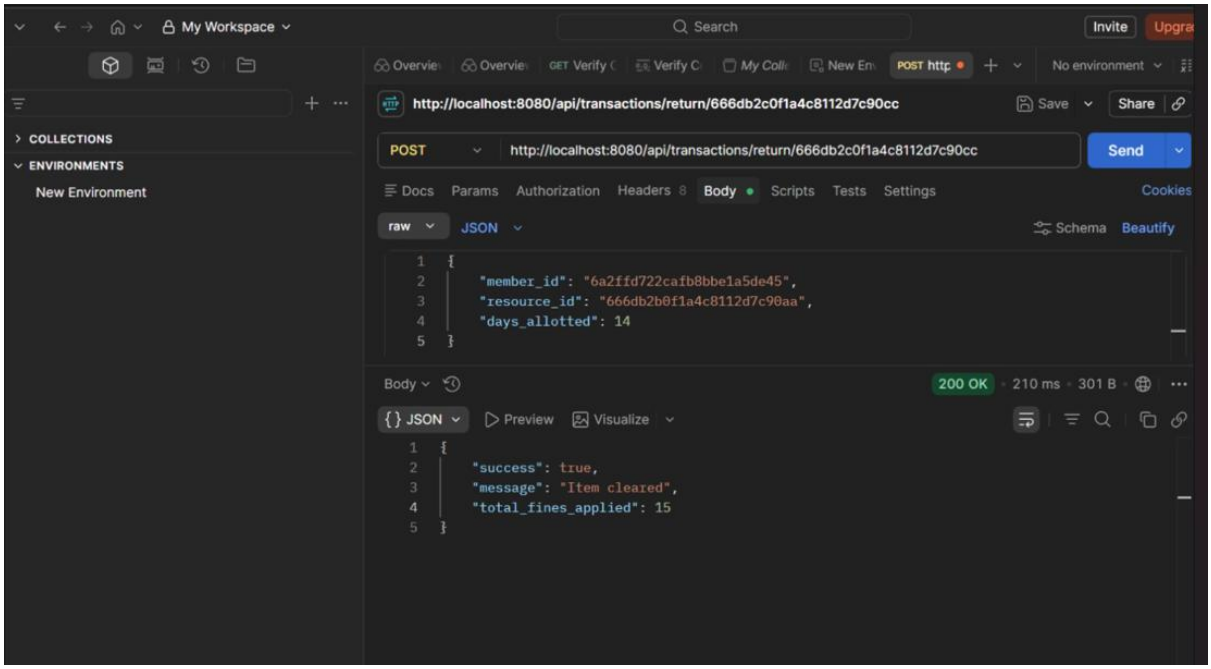
File Reference:

- **Operational Significance:** Validates the resource borrow transaction pipeline via a POST request to /api/transactions/borrow. The engine successfully decrements current inventory stock levels and issues an active due-date return string.
- **File Reference:**



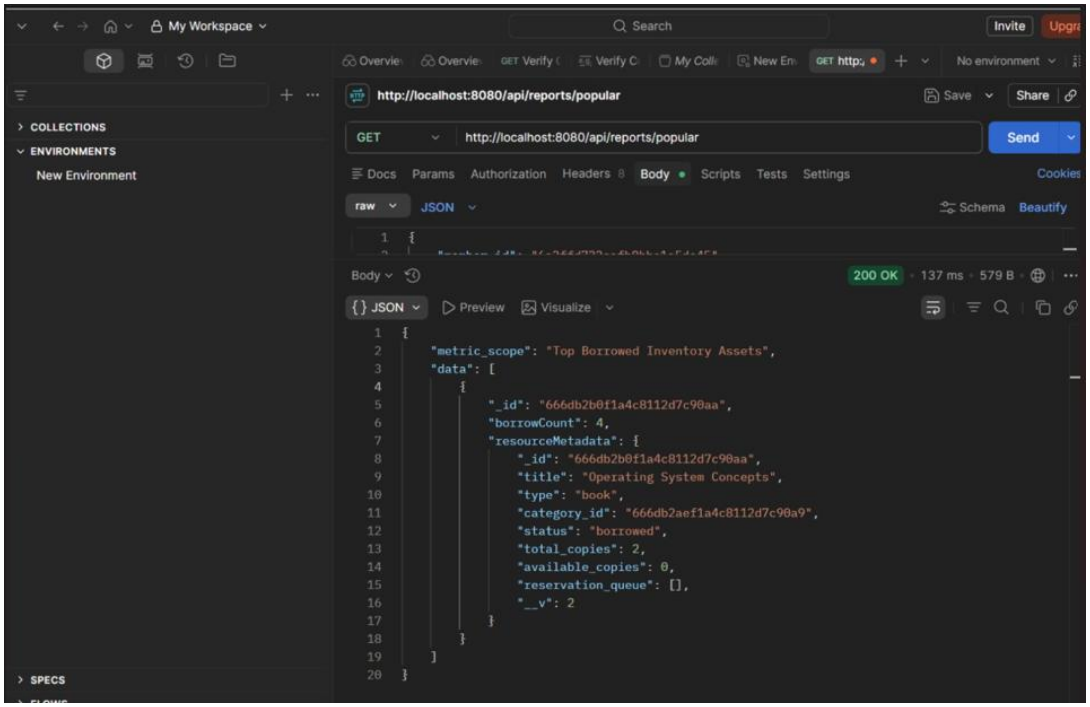
- **Operational Significance:** Proves concurrency waitlist handling. Resending a checkout request when item availability hits zero returns a 202 Accepted status code, verifying that the member was cleanly appended to the FIFO reservation queue.

- **File Reference:**



- **Operational Significance:** Validates item returns and business rules via an HTTP POST to the return route. It processes overdue transaction intervals, clears the asset lock, and outputs computed fine parameters.

- **File Reference:**



- **Operational Significance:** Demonstrates advanced administrative data analytics via GET /api/reports/popular. The output confirms a successful native database lookup aggregation run across multiple collections.

4. CONCLUSION & ARTIFACT REPOSITORY

The complete structural code repository has been compressed into a clean production package (LLRMS_Final_Submission.zip) and uploaded to GitHub.

The development of the LLRMS platform successfully demonstrates the efficiency of merging a NoSQL database with an asynchronous JavaScript runtime. Every transactional route executed with stable data state transformations and accurate chronological boundary checks. Future updates can extend functionality by implementing JSON Web Token (JWT) user authentication and real-time frontend notifications via WebSockets.